CyberDefense Challenge: Cisco EDGE Cybersecurity CTF Writeup

Author: Imane Habbani (player: imhvb)
Event: Cisco EDGE Cybersecurity CTF, individual format, 90 minutes
Date: 2 November 2026

Overview

The CyberDefense Challenge was a 35-flag mission aimed at Cisco Networking Academy learners preparing for CCST Cybersecurity roles. It covered security fundamentals, Cisco IOS and Snort, CVSS scoring, a sandbox report analysis and packet capture forensics.

This writeup records what each question asked and how I got to the answer. I have left out all flag values so the challenges remain usable for other participants. The organizers supplied every target and file for authorized, educational testing, and AI assistance was explicitly permitted.

Approach and tooling. With only 90 minutes, I worked in order of effort. Conceptual questions came first, at roughly a minute each. Anything that stalled after a few attempts was parked, and the longer analysis tasks were saved for when I could give them proper time. I used a browser for the CTF platform and the ANY.RUN report, and tshark on Kali Linux together with base64 and xxd for the packet analysis.

Result. 34 of 35 flags solved. Flag 14 is the only one I left open (see section 5).

1. Security Fundamentals (flags 1-13, 15, 16)

These questions test whether you can map a description to the exact term the course uses. Most of the effort goes into reading what the question wants: a name, a command, a number or a specific phrase format.

Flag 1, Terms and Conditions. A free-points task. The linked document sat behind a password prompt, but the task text already stated what to enter, so I followed it as written.

Flag 2, Defense Strategies. The question describes concealing the operating system and servers in use so nobody can learn what is running. That is the principle of protecting a system by hiding information about it, and the task was to recall its standard name.

Flag 3, Mitigation Techniques. "Secure access to an internal network from any device and location" describes the technology that builds an encrypted, authenticated tunnel for remote users.

Flag 4, Check Digit Calculation. A checksum that can be done by hand. Multiply the ten digits by weights from 10 down to 1, add the products, and check that the total is divisible by 11.

Flag 5, OWASP Foundation. An invisible button placed over a visible one tricks the user into clicking the wrong thing. The transparent layer over a legitimate control is what defines this particular OWASP attack class.

Flag 6, Linux Access Control. The NSA designed the SHA family, but the question asked for one hash function, and the family name was rejected as too generic. The accepted answer was the specific named function from that family that the course material ties to the NSA.

Flag 7, Configure ZPF. In Cisco IOS Zone-Based Firewall, zones are created globally but an interface joins a zone from interface configuration mode. The answer is therefore an interface-level command followed by the zone name.

Flag 8, Cloud-Based Technology. I compared the three service models (infrastructure, platform, software) and picked the one defined by subscription access to development tools and services.

Flag 9, Proxy Logs. The image shows a single Squid access-log line. In that format the size delivered to the client comes after the cache result and HTTP status. The field before it is elapsed time, which is the easy one to confuse with size.

Flag 10, Snort and Firewall Rules. The image is a terminal capture of a download command. The port is part of the URL in the command, and the connection output underneath confirms it.

Flag 11, The PKI Authorities System. Certificate classes are numbered by assurance level, with higher numbers meaning stricter verification. The most trusted class is the highest one.

Flag 12, Encrypting and Decrypting Data Using a Hacker Tool. Matched from the toolset used in the course labs: the utility built specifically for recovering passwords from zip archives.

Flag 13, Access Control Lists. First identify the ICMP message generated when a packet's TTL reaches 0, then recall the IOS keyword for that message type. The row of X characters in the question is only a placeholder and says nothing about the keyword's length.

Flag 15, Steps of the Cyber Kill Chain. Spear phishing is classically associated with the Delivery phase, and that was my first answer, but several attempts were rejected. The question emphasises attachments designed to install malware when opened, which points to the phase in which the malware is installed on the victim. The lesson: answer what the question stresses, not the textbook association.

Flag 16, Cyber Kill Chain. The partial list had five steps, with a gap after the second and another after the fourth. I compared it against the standard seven-step Cyber Kill Chain, identified the step missing from each gap and wrote them in the requested "Step and Step" format.

2. Vulnerability Scoring (flags 17-18)

Both flags give a vulnerability description and ask for the CVSS v3.1 base score. My method was the same each time: classify the attack, choose each base metric from what the text actually states, then compute the score with the v3.1 equations. Both descriptions also match worked examples in FIRST's CVSS examples document, which served as a cross-check.

Flag 17, MySQL Server database. A SQL injection that abuses an identifier in a database configured for replication. The attacker needs a valid low-privilege account and network access, with no user interaction. The injected SQL is later executed on remote replica servers, which sit under a different security authority, so the scope changes. Impact is limited for confidentiality and integrity, and none for availability.

Flag 18, Virtual Machine Executable (VMX). A flaw in the RPC handler of the VMX process. A guest user sends crafted RPC commands over the network, needs only a guest account, and can escape the guest to affect the host, so the scope changes here as well. Confidentiality, integrity and availability impact are all high.

3. Sandbox Report Analysis (flags 19-26)

Flag 19 only asks you to confirm you have read the instructions (do not download the sample, no account needed). Flags 20 to 26 all use the same public ANY.RUN report on a malicious email attachment. I worked entirely from the report and never downloaded or executed the sample.

Flag 20, Malicious file. The submitted object is named in the report header. I copied it exactly, including its double extension.

Flag 21, The Hash. The MD5 sits under the file name in the same header. I copied it from the page instead of retyping it.

Flag 22, The Trojan Horse. The family name appears in the header tags and in the Tracker line.

Flag 23, The Process. Results, then Process graph. Find the cmd.exe node and read its parent, which is highlighted in red.

Flag 24, The MITRE ATT&CK. Results, then the ATT&CK matrix. The header shows the three counters in the order the answer format needs (tactics, techniques, events). The matrix loaded blank at first; closing it, reloading the report and reopening it fixed that.

Flag 25, The Tactics. These are the highlighted columns of the matrix.

Flag 26, Dangerous Technique. The matrix legend counts items by severity, so I looked for the technique highlighted in red.

4. Packet Capture Analysis (flags 27-35)

The capture came as a password-protected zip. I extracted it on Kali and used tshark throughout, since its output is easy to reproduce and pipe into other tools.

```
cd ~/Downloads && unzip -P '<password>' sample.zip   # single quotes stop the
shell expanding $
tshark -r sample.pcap | head
```

Flag 27, PCAP File. Confirmation only. The task also points to a lab VM holding the file, but extracting the zip locally worked just as well.

Flag 28, HTTP. The question asks how many HTTP packets (requests and responses combined) the capture contains. The total depends on how HTTP is counted, so I compared several methods: the plain `http` filter, request and response filters, the `ssdp` filter, and the per-method and per-status breakdowns. The capture also carries HTTP-style messages over UDP (SSDP discovery uses NOTIFY and M-SEARCH), so counting TCP alone misses part of the total. I kept reconciling the filters until two methods agreed.

```
tshark -r sample.pcap -Y http | wc -l
tshark -r sample.pcap -Y "http.request || http.response" | wc -l
tshark -r sample.pcap -Y ssdp | wc -l
tshark -r sample.pcap -q -z io,phs | grep -i -E "http|ssdp"
tshark -r sample.pcap -q -z http,stat
```

Flag 29, Ports. I counted destination ports and sorted by frequency. The difficulty is in reading the question: it could mean the busiest destination port by packet count, or the busiest service port once client-side ephemeral ports are discarded. I filtered out ephemeral ports first, which was the wrong reading here; the accepted answer came from the raw packet counts. Comparing packet counts with SYN counts per port is a good way to see both views side by side.

```
tshark -r sample.pcap -T fields -e tcp.dstport -e udp.dstport \
  | tr '\t' '\n' | grep -v '^$' | sort | uniq -c | sort -nr | head
```

Flag 30, IPs. I looked at the busiest conversation on a non-standard port and listed its endpoints. The internal host is the one in the private range. The external host it keeps talking to over HTTP is the attacker's server, which the beacon traffic in the later flags confirms.

```
tshark -r sample.pcap -Y "tcp.port==<suspect_port>" -T fields \
  -e ip.src -e ip.dst -e tcp.srcport -e tcp.dstport | sort | uniq -c | sort -
nr | head
```

Flags 31 and 32, First Take. Both flags ask for the filename of the first file the infected host downloaded from the threat actor's server. I listed the HTTP requests to the attacker's server in frame order and found the first download. Its URI is only an endpoint, so the filename had to come from the server's response headers, such as Content-Disposition. The file was never executed.

```
tshark -r sample.pcap -Y "ip.addr==<attacker_ip> && http.request" -T fields \
  -e frame.number -e http.request.method -e http.host -e http.request.uri |
head
tshark -r sample.pcap -Y "http.response && ip.src==<attacker_ip>" -T fields \
  -e frame.number -e http.response.line | head
```

Flag 33, Diana. The question asks which C2 frameworks the adversaries might be using. Three indicators together pointed to one: the name of the downloaded agent, the regular POST requests to a beacon endpoint, and the server header. Each matches the public documentation of a specific adversary-emulation framework.

Flag 34, 47 not 007. The question asks for the agent ID the C2 agent uses. Beacon bodies appear as hex in tshark output, base64 inside that, and JSON underneath. I decoded every beacon, extracted the identifier field and counted its values. The empty value belongs to the first check-in, before the server assigns an ID, and the value used by all later beacons is the agent ID.

```
tshark -r sample.pcap -Y 'http.request.uri=="/beacon"' -T fields -e
http.file_data \
  | while read l; do
      echo $l | xxd -r -p | base64 -d 2>/dev/null | grep -o '"paw":"[^"]*"'
    done | sort | uniq -c
```

Flag 35, The Line. The question asks for the first command the C2 server sent to the infected host. Responses to the beacon are base64 JSON with an instructions field, and each command inside is base64-encoded a second time. I decoded the responses in frame order and read the first one carrying an instruction. My first attempts were rejected and I used a hint. The practical lesson is to decode each layer in turn and, if a decoded value is rejected, to try the string exactly as it appears in the traffic.

```
tshark -r sample.pcap -Y "http.response && ip.src==<attacker_ip>" -T fields \
  -e frame.number -e http.file_data \
  | while read n d; do echo "== $n"; echo $d | xxd -r -p | base64 -d; echo;
done
```

5. The Unsolved Flag (flag 14)

Flag 14 presents a Snort rule for a possible SQL injection and asks which exact part of the output shows which element of the IP packet caused the alert. My reasoning was that the rule header (protocol, addresses, ports) only defines where the rule applies, that `msg` is a human-readable label, and that the `content` options match payload bytes such as the HTTP method, URI and form fields. Several answers built on that idea were rejected, including after I took the hint, so I moved on to protect my time for the packet analysis. It was the only flag I left unsolved.

6. Takeaways

1. **Answer what the question emphasises, not the textbook association.** Flags 6 and 15 both turned on this.
2. **Test more than one reading of an ambiguous question before committing.** The port count and the HTTP total both needed it, and protocols such as HTTP-style discovery traffic can also run over UDP.
3. **Decode every layer of C2 traffic.** The useful data was hidden in encodings nested inside each other.

Throughout the event, the sandbox report and the packet capture provided all the evidence needed without ever running the malware.

All targets, files and the sandbox report were provided by the organizers for authorized, educational testing.